```
Input: input

Output: MobilenetV1/Predictions/Reshape_1
```

Step 3: Importing and compiling

The latest version of TVM provides a sub-module relay (*tvm.relay*) which contains all the frontend import utilities. Please refer to the code snippet given below to import and build the TensorFlow model on TVM.

```
# import tvm and tensorflow
import tvm
import tensorflow as tf

# We want to build TVM for llvm target (x86)
target = 'llvm'

# Import the tensorflow model.
with tf.gfile.FastGFile(os.path.join("mobilenet_v1_1.0_224_
frozen.pb"), 'rb') as f:
    graph_def = tf.GraphDef()
    graph_def.ParseFromString(f.read())
    graph = tf.import_graph_def(graph_def, name='')

    # Call the utility to import the graph definition into
default graph.
    graph_def = tf_testing.ProcessGraphDefParam(graph_def)



    # Add shapes to the graph.
    # Alternatively can use "add_shapes=True" while exporting
graph from Tensorflow.
    with tf.Session() as sess:
        graph_def = tvm.relay.testing.
tf.AddShapesToGraphDef(sess, 'MobilenetV1/Predictions/
Reshape_1')

# Set input shape for graph input.
shape_dict = {'input': (1, 244, 244, 3)}

# Import graph through frontend
sym, params = relay.frontend.from_tensorflow(graph_def,
shape=shape_dict)

# sym : represents tvm symbol graph constructed from imported
model.

# params : graph parameters imported from model.

# Compile the model on TVM.
# The target here indicates the compiler to build the output
for 'llvm'
graph, lib, params = relay.build(sym, target=target,
params=params)
```

Step 4: Saving the compiler output

In the above step, the build process has resulted in a graph, *lib* and *params*. The graph is an object which holds the compiler graph. *lib* represents the library for the 'llvm' target and *params* represents the parameters for the model. Please refer to the code snippet below to save the compilation output to the disk.

```
# nnvm is part of TVM which is old version of compiler.
# we use the save_param_dict from here.
import nnvm

# Save the model as a library.
lib.export_library("libmobilenet.so")

# Save the graph definition as a JSON.
with open("mobilenet.json", "w") as fo:
    fo.write(graph.json())

# Save the params.
with open("mobilenet.params", "wb") as fo:
    fo.write(nnvm.compiler.save_param_dict(params))
```

Step 5: Loading and executing

The saved outputs from the compilation process are a library, a JSON and a *params* binary file. We may take these executables as targets for deploying data files. Along with these, we need a target-specific application to load the compiled model on the target. In our case, the target is x86 and for the deployment explanation I am choosing Python. The code snippet which follows with inlined documentation shows a Python application to deploy and infer.

```
# Load the module
loaded_lib = tvm.module.load("libmobilenet.so")
# Read graph and params.

loaded_json = open("mobilenet.json").read()
loaded_params = bytearray(open("mobilenet.params", "rb").
read())

# graph runtime initializes runtime from loaded module,
# graph and on given context. In this case the context is
CPU.
from tvm.contrib import graph_runtime
module = graph_runtime.create(loaded_json, loaded_lib, tvm.
cpu(0))

# Initialize the parameters.
params = nnvm.compiler.load_param_dict(loaded_params)
module.load_params(loaded_params)

# Initialize some random data for model input.
input_data = np.random.uniform(size=(1, 244, 244, 3)).
```